

Week13_복습과제_백재은

Ch.5 콘텐츠 기반 필터링 실습 - TMDb 5000 영화 데이터 세트

1. 장르 속성을 이용한 영화 콘텐츠 기반 필터링

콘텐츠 기반 필터링: 사용자가 특정 영화를 감상하고 그 영화를 좋아했다면 그 영화와 비슷한 특성/속성, 구성 요소 등을 가진 다른 영화를 추천하는 것

Topic → 콘텐츠 기반 필터링 추천 시스템을 영화를 선택하는 데 중요한 요소인 영화 장르 속성을 기반으로 구축, 장르 칼럼 값의 유사도 비교, 높은 평점을 가지는 영화 추천 방식

2. 데이터 로딩 및 가공

```
import pandas as pd
import numpy as np
import warnings; warnings.filterwarnings('ignore')

movies = pd.read_csv('./tmdb-5000-movie-dataset/tmdb_5000_movies.csv')
print(movies.shape)
movies.head(1)
```

[Output]

(4803, 20)

	budget	genres	homepage	id	keywords	original_language	original_title	overview	popularity	pr...
0	237000000	["id": 28, "name": "Action", "id": 12, "nam...	http://www.avatarmovie.com/	19995	["id": 1463, "name": "culture clash", "id": ...	en	Avatar	In the 22nd century, a paraplegic Marine is d...	150.437577	

tmdb_5000_movies.csv 파일 df로 로딩, 데이터 살펴보기

4803 레코드, 20개 피쳐

주요 칼럼 추출하여 new DF 생성 → genres, vote_average, vote_count, popularity, keywords, overview

Genres, keywords → 리스트 내부에 여러 개 딕셔너리가 있는 형태의 문자열 → 분해해서 개별 장르를 파이썬 리스트 객체로 추출

```
from ast import literal_eval
movies_df['genres'] = movies_df['genres'].apply(literal_eval)
movies_df['keywords'] = movies_df['keywords'].apply(literal_eval)
```

→ 리스트 내부에 여러 장르 딕셔너리로 구성된 객체

```
movies_df['genres'] = movies_df['genres'].apply(lambda x : [ y['name'] for y in x])
movies_df['keywords'] = movies_df['keywords'].apply(lambda x : [ y['name'] for y in x])
movies_df[['genres', 'keywords']]
```

→ apply lambda 식 활용, 장르명만 리스트 객체로 추출

3. 장르 콘텐츠 유사도 측정

단계 → 문자열 변환 genres 칼럼을 Count 기반으로 피쳐 벡터화 변환 / genres 문자열을 피쳐 벡터화 행렬로 변환한 데이터 세트를 코사인 유사도로 비교 (레코드별로 코사인 유사도 값을 가지는 객체 생성) / 장르 유사도가 높은 영화 중 평점이 높은 순으로 영화 추천

a. 피쳐 벡터 행렬 생성 → CounterVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer

# CountVectorizer를 적용하기 위해 공백문자로 word 단위가 구분되는 문자열로 변환.
movies_df['genres_literal'] = movies_df['genres'].apply(lambda x : ' '.join(x))
count_vect = CountVectorizer(min_df=0, ngram_range=(1, 2))
genre_mat = count_vect.fit_transform(movies_df['genres_literal'])
print(genre_mat.shape)
```

[Output]

(4803, 276)

b. 코사인 유사도 계산
→ cosine_similarity()

기준 행	0	1	7	2
	1	1	2	4
	2	0	8	3
	3	2	0	3

↓ cosine_similarities(적용)

비교 대상 행				
	0	1	2	3
0	1 (0행 자신과의 유사도)	0.68 (0행과 1행의 유사도)	0.99 (0행과 2행의 유사도)	0.3 (0행과 3행의 유사도)
1	0.68 (1행과 0행의 유사도)	1 (1행 자신과의 유사도)	0.72 (1행과 2행의 유사도)	0.85 (1행과 3행의 유사도)
2	0.99 (2행과 0행의 유사도)	0.72 (2행과 1행의 유사도)	1 (2행 자신과의 유사도)	0.29 (2행과 3행의 유사도)
3	0.3 (3행과 0행의 유사도)	0.85 (3행과 1행의 유사도)	0.29 (3행과 2행의 유사도)	1 (3행 자신과의 유사도)

c. 장르 유사도가 높은 순으로 다른 레코드 추출
→ genre_sim 객체 이용
→ argsort()[::-1] - 유사도가 높은 순으로 정렬된 객체의 비교 행 위치 인덱스 값

4. 장르 콘텐츠 필터링을 이용한 영화 추천
→ fine_sim_movie(): 장르 유사도에 따라 영화를 추천하는 함수
→ 기반 데이터, 유사도 인덱스, 영화 제목, 영화 건수 입력 시 추천 영화 DF 반환

a. 대부와 장르별로 유사한 영화 10개 추천

```
similar_movies = find_sim_movie(movies_df, genre_sim_sorted_ind, 'The Godfather', 10)
similar_movies[['title', 'vote_average']]
```

[Output]

```
[[2731 1243 3636 1946 2640 4065 1847 4217 883 3866]]
```

	title	vote_average
2731	The Godfather: Part II	8.3
1243	Mean Streets	7.2
3636	Light Sleeper	5.7
1946	The Bad Lieutenant: Port of Call - New Orleans	6.0
2640	Things to Do in Denver When You're Dead	6.7
4065	My America	0.0
1847	GoodFellas	8.2
4217	Kids	6.8
883	Catch Me If You Can	7.7
3866	City of God	8.1

b. 영화의 평점에 따라 필터링 후 최종 추천 방식으로 변경

→ vote_average 값 이용
→ 적은 수의 관객이 높은 값을 줬서 왜곡된 데이터

```
movies_df[['title', 'vote_average', 'vote_count']].sort_values('vote_average', ascending=False)[:10]
```

[Output]

	title	vote_average	vote_count
3519	Stiff Upper Lips	10.0	1
4247	Me You and Five Bucks	10.0	2
4045	Dancer, Texas Pop. 81	10.0	1
4662	Little Big Top	10.0	1
3992	Sendaarji	9.5	2
2386	One Man's Hero	9.3	2
2970	There Goes My Baby	8.5	2
1881	The Shawshank Redemption	8.5	8205
2796	The Prisoner of Zenda	8.4	11
3337	The Godfather	8.4	5893

c. 평가 횟수에 대한 가중치가 부여된 평점 방식 이용

가중 평점(Weighted Rating) = $(v/(v+m)) * R + (m/(v+m)) * C$

V: vote_count / R: vote_average / C: movies_df['vote_average'].mean() / m: Series 객체의 quantile()

→ weighted_vote 평점이 높은 순으로 상위 10개 영화 추출

```
movies_df[['title', 'vote_average', 'weighted_vote', 'vote_count']].sort_values(
    'weighted_vote', ascending=False)[:10]
```

[Output]

	title	vote_average	weighted_vote	vote_count
1881	The Shawshank Redemption	8.5	8.398052	8205
3337	The Godfather	8.4	8.263501	5893
662	Fight Club	8.3	8.210455	9413
3232	Pulp Fiction	8.3	8.207102	8428
65	The Dark Knight	8.2	8.136930	12002
1818	Schindler's List	8.3	8.126069	4329
3865	Whiplash	8.3	8.123248	4254
809	Forrest Gump	8.2	8.105954	7927
2294	Spirited Away	8.3	8.105867	3840
2731	The Godfather: Part II	8.3	8.079586	3338

d. 장르 유사성이 높은 영화를 top_n의 2배수만큼 후보군으로 선정, weighted_vote 칼럼 값이 높은 순으로 top_n만큼 추출

→ fine_sim_movie() 함수 변경

```
def find_sim_movie(df, sorted_ind, title_name, top_n=10):
    title_movie = df[df['title'] == title_name]
    title_index = title_movie.index.values

    # top_n의 2배에 해당하는 장르 유사성이 높은 인덱스 추출
    similar_indexes = sorted_ind[title_index, :(top_n*2)]
    similar_indexes = similar_indexes.reshape(-1)
    # 기존 영화 인덱스는 제외
    similar_indexes = similar_indexes[similar_indexes != title_index]

    # top_n의 2배에 해당하는 후보군에서 weighted_vote가 높은 순으로 top_n만큼 추출
    return df.iloc[similar_indexes].sort_values('weighted_vote', ascending=False)[:top_n]

similar_movies = find_sim_movie(movies_df, genre_sim_sorted_ind, 'The Godfather', 10)
similar_movies[['title', 'vote_average', 'weighted_vote']]
```

[Output]

	title	vote_average	weighted_vote
2731	The Godfather: Part II	8.3	8.079586
1847	GoodFellas	8.2	7.978937
3865	City of God	8.1	7.759693
1663	Once Upon a Time in America	8.2	7.657811
883	Catch Me if You Can	7.7	7.557097
281	American Gangster	7.4	7.141396
4041	This Is England	7.4	6.739664
1149	American Hustle	6.8	6.717525
1243	Mean Streets	7.2	6.620509
2839	Rounders	6.9	6.530427

Ch.6 아이템 기반 최근접 이웃 협업 필터링 실습

1. 협업 필터링: 사용자와 아이템 간의 평점에 기반해 추천하는 시스템

로우 레벨의 형태의 원본 데이터 세트 → 모든 사용자를 로우, 모든 영화를 칼럼으로 구성한 데이터 세트로 변경 / DF의 pivot_table() 함수 이용

로우 레벨 형태의 사용자-영화 평점 데이터

userId	movieId	rating
User 1	Movie1	3
User 1	Movie3	3
User 1	Movie6	4
User 2	Movie2	1
User 2	Movie4	5
User 3	Movie5	4

변환

사용자 로우, 영화 칼럼으로 구성된
사용자-영화 평점 데이터

	Movie 1	Movie 2	Movie 3	Movie 4	Movie 5	Movie 6
User 1	3		3			4
User 2		1		5		
User 3					4	

→ NaN 값이 많음
→ 사용자가 평점을 매기지 않은 영화가 칼럼으로 변환되면서...

```
ratings = ratings[['userId', 'movieId', 'rating']]
ratings_matrix = ratings.pivot_table('rating', index='userId', columns='movieId')
ratings_matrix.head(3)
```

[Output]

movieId	1	2	3	4	5	6	7	8	9	10	...	193565	193567	193571	193573	193579	193581	193583	193585	193587	193609
userId																					
1	4.0	NaN	4.0	NaN	NaN	4.0	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

→ 칼럼명을 title로 변경, NaN 0 변환

[Output]

	Verity: The Young Samuels' Season 1 (2014)			'Tis the Season: The Young Samuels' Season 1 (2014)			'Tis the Season: The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season 1 (2014)			The Young Samuels' Season		
--	--	--	--	---	--	--	---	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	------------------------------------	--	--	---------------------------	--	--

a. 영화 간 유사도 산출

→ cosine_similarity() 이용 측정, 현재의 ratings_matrix가 행 기준: 영화 / 열 기준: 사용자
→ 전치 행렬 변경: transpose() 함수
→ 직관적인 유사도 값 표현: cosine_similarity() 반환 넘파이 행렬에 영화명 매핑, DF로 변환

```
ratings_matrix_T = ratings_matrix.transpose()
ratings_matrix_T.head(3)
```

[Output]

userId	1	2	3	4	5	6	7	8	9	10	...	601	602	603	604	605	606	607	608	609	610
title																					
'71 (2014)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

title	'71 (2014)	'Hellboy: The Seeds of Creation (2004)	'Round Midnight (1986)	'Salem's Lot (2004)	'There Was a Little Girl (1997)	'Tis the Season for the Mother (1986)	'Turks, Cabaret, and the Mother (1986)	'(500) Days of Summer (2009)	'batteries not included (1987)	Zulu (2013)
'71 (2014)	1.0	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.0	0.141653	0.0
'Hellboy: The Seeds of Creation (2004)	0.0	1.000000	0.707107	0.0	0.0	0.0	0.000000	0.0	0.000000	0.0
'Round Midnight (1986)	0.0	0.707107	1.000000	0.0	0.0	0.0	0.176777	0.0	0.000000	0.0

→ item_sim을 DataFrame으로 변환한 item_sim_df를 이용해 영화 '대부(Godfather, The (1972))'와 유사도가 높은 상위 6개 영화를 추출

```
item_sim_df[["Godfather, The (1972)"]].sort_values(ascending=False)[6]
```

[Output]

title	
Godfather, The (1972)	1.000000
Godfather: Part II, The (1974)	0.821773
Goodfellas (1990)	0.664841
One Flew Over the Cuckoo's Nest (1975)	0.620536
Star Wars: Episode IV - A New Hope (1977)	0.595317
Fargo (1996)	0.588614

→ 인셉션에 적용

```
item_sim_df[["Inception (2010)"]].sort_values(ascending=False)[16]
```

[Output]

title	
Dark Knight, The (2008)	0.727263
Inglourious Basterds (2009)	0.646183
Shutter Island (2010)	0.617736
Dark Knight Rises, The (2012)	0.617594
Fight Club (1999)	0.615417

2. 아이템 기반 최근접 이웃 협업 필터링으로 개인화된 영화 추천

$$\hat{R}_{u,i} = \sum^N (S_{i,N} * R_{u,N}) / \sum^N (|S_{i,N}|)$$

변수의 의미:

- $\hat{R}_{u,i}$: 사용자 u , 아이템 i 의 개인화된 예측 평점 값
- $S_{i,N}$: 아이템 i 와 가장 유사도가 높은 Top-N개 아이템의 유사도 벡터
- $R_{u,N}$: 사용자 u 의 아이템 i 와 가장 유사도가 높은 Top-N개 아이템에 대한 실제 평점 벡터

N값: 아이템의 최근접 이웃 범위 계수, 특정 아이템과 유사도가 가장 높은 Top-N개의 다른 아이템을 추출하는 데 사용

사용자별 영화 예측 평점 R: 사용자의 모든 영화에 대한 실제 평점과 영화 i 의 다른 모든 영화와의 코사인 유사도를 벡터 내적 곱한 값을 정규화를 위해 나눈 것

a. predict_rating() 함수를 통해 개인화된 예측 평점 구하기

→ ratings_matrix, item_sim_df를 넘파이 행렬로 변환, 인자 입력

→ 예측 평점이 실제 평점에 비해 작을 수 있음 - 코사인 유사도 벡터 합으로 나눠서

b. 예측 결과와 실제 평점과 갖는 차이 확인

→ MSE 적용, get_mse() 함수 생성 및 결과 확인

→ MSE 감소 방향으로 개선해야 함

```
from sklearn.metrics import mean_squared_error

# 사용자가 평점을 부여한 영화에 대해서만 예측 성능 평가 MSE를 구함.
def get_mse(pred, actual):
    # 평점이 있는 실제 영화만 추출
    pred = pred[actual.nonzero()].flatten()
    actual = actual[actual.nonzero()].flatten()
    return mean_squared_error(pred, actual)

print('아이템 기반 모든 최근접 이웃 MSE: ', get_mse(ratings_pred, ratings_matrix.values))
```

[Output]

아이템 기반 모든 최근접 이웃 MSE: 9.895354759094766

c. 특정 영화와 가장 비슷한 유사도를 가지는 영화에 대해서만 유사도 벡터를 적용하는 함수로 변경

→ predict_rating_topism(ratings_arr, item_sim_arr, n=20) 함수

→ 개별 예측값을 구하기 위해 루프를 반복 수행하여 수행 시간 길

→ MSE: 3.69로 개선

```
def predict_rating_topism(ratings_arr, item_sim_arr, n=20):
    # 사용자-아이템 평점 행렬 크기만큼 0으로 채운 예측 행렬 초기화
    pred = np.zeros(ratings_arr.shape)

    # 사용자-아이템 평점 행렬의 열 크기만큼 루프 수행.
    for col in range(ratings_arr.shape[1]):
        # 유사도 행렬에서 유사도가 큰 순으로 n개 데이터 행렬의 인덱스 반환
        top_n_items = [np.argsort(item_sim_arr[:, col])[-n-1:-1]]
        # 개인화된 예측 평점을 계산
        for row in range(ratings_arr.shape[0]):
            pred[row, col] = item_sim_arr[col, :][top_n_items].dot(ratings_arr[row, :][top_n_items].T)

        pred[row, col] /= np.sum(np.abs(item_sim_arr[col, :][top_n_items]))

    return pred
```

d. 특정 사용자에게 대해 영화 추천

→ user_id = 9

→ 평점 높은 순으로 나열

```
user_rating_id = ratings_matrix.loc[9, :].sort_values(ascending=False)[:10]
```

[Output]

title	
Adaptation (2002)	5.0
Citizen Kane (1941)	5.0
Raiders of the Lost Ark (Indiana Jones and the Raiders of the Lost Ark) (1981)	5.0
Producers, The (1968)	5.0
Lord of the Rings: The Two Towers, The (2002)	5.0
Lord of the Rings: The Fellowship of the Ring, The (2001)	5.0
Back to the Future (1985)	5.0
Austin Powers in Goldmember (2002)	5.0
Minority Report (2002)	4.0
Witness (1985)	4.0

E. 아이템 기반 협업 필터링을 통해 영화 추천

→ get_unseen_movies() 생성 / 평점을 준 영화 제외 추천, 평점 안 준 영화 리스트 객체로 반환

→ 영화 추천 함수 `recomm_movie_by_userid()` 생성

→ 해당 함수는 예측 평점 DataFrame과 추천하려는 사용자id, 추천 후보 영화 리스트, 추천 상위 영화 개수를 인자로 받아서 사용자가 좋아할 만한 가장 높은 예측 평점을 가진 영화를 추천

Ch.7 행렬분해를 이용한 잠재 요인 협업 필터링 실습

행렬 분해 잠재 요인 협업 필터링: SVD, NMF 적용

→ 사용자 - 아이템 평점 행렬: 사용자가 평점을 매기지 않은 널 데이터 많음, SGD/ALS 기반 행렬 분해

→ SGD 기반 행렬 구현 기반 사용자 영화 추천

`matrix_factorization` 함수 파라미터 정리

- R: 사용자-아이템 평점 행렬 (원본 데이터)
- K: 잠재 요인(latent factor)의 차원 수
- steps: 경사 하강법(SGD)의 반복 횟수 (기본값: 200)
- learning_rate: 학습률 (기본값: 0.01)
- r_lambda: L2 정규화(규제) 계수 (기본값: 0.01)

```
def matrix_factorization(R, K, steps=200, learning_rate=0.01, r_lambda=0.01):
    num_users, num_items = R.shape
    # P와 Q 매트릭스의 크기를 지정하고 정규 분포를 가진 랜덤한 값으로 입력합니다.
    np.random.seed(1)
    P = np.random.normal(scale=1./K, size=(num_users, K))
    Q = np.random.normal(scale=1./K, size=(num_items, K))

    # R > 0 인 행 위치, 열 위치, 값을 non_zeros 리스트 객체에 저장.
    non_zeros = [(i, j, R[i, j]) for i in range(num_users) for j in range(num_items) if R[i, j] > 0]

    # SGD기법으로 P와 Q 매트릭스를 계속 업데이트.
    for step in range(steps):
        for i, j, r in non_zeros:
            # 실제 값과 예측 값의 차이인 오류 값 구함
            eij = r - np.dot(P[i, :], Q[j, :].T)
            # Regularization을 반영한 SGD 업데이트 공식 적용
            P[i, :] = P[i, :] + learning_rate*(eij * Q[j, :] - r_lambda*P[i, :])
            Q[j, :] = Q[j, :] + learning_rate*(eij * P[i, :] - r_lambda*Q[j, :])

        rmse = get_rmse(R, P, Q, non_zeros)
        if (step % 10) == 0:
            print("### iteration step : ", step, " rmse : ", rmse)

    return P, Q
```

```
import pandas as pd
import numpy as np

movies = pd.read_csv('./ml-latest-small/movies.csv')
ratings = pd.read_csv('./ml-latest-small/ratings.csv')
ratings = ratings[['userId', 'movieId', 'rating']]
ratings_matrix = ratings.pivot_table('rating', index='userId', columns='movieId')

# title 칼럼을 얻기 위해 movies와 조인 수행
rating_movies = pd.merge(ratings, movies, on='movieId')
```

→ 영화 평점 행렬 데이터를 DF로 로딩, 다시 사용자 - 아이템 평점 행렬

b. 사용자 - 아이템 평점 행렬을 `matrix_factorization()` 함수로 행렬 분해

→ steps: 200, K: 50, L2 Regularization/학습률: 0.01

```
P, Q = matrix_factorization(ratings_matrix.values, K=50, steps=200, learning_rate=0.01,
                             r_lambda=0.01)
pred_matrix = np.dot(P, Q.T)
```

[Output]

```
### iteration step : 0      rmse : 2.9623619751336867
### iteration step : 10     rmse : 0.7335768591817927
.....
### iteration step : 190    rmse : 0.1488947991323209
```

c. 영화 타이틀을 칼럼명으로 가지는 DF로 변경

```
ratings_pred_matrix = pd.DataFrame(data=pred_matrix, index=ratings_matrix.index,
                                   columns=ratings_matrix.columns)
ratings_pred_matrix.head(3)
```

	'Hellboy': 71 (2014)	'The Seeds of Creation (2004)	'Round Midnight (1999)	'Salem's Lot (2004)	'Til There Was You (1997)	'Tis the Season for Love (2015)	'burbs. The (1989)	'night Mother (1986)	(500) Days of Summer (2009)	'batteries not included (1987)	...	Zulu (2013)
userid												
1	3.055084	4.082018	3.564130	4.502167	3.981215	1.271994	3.603274	2.333266	5.091749	3.672854	...	1.402608
2	3.170119	3.657992	3.308707	4.160521	4.311890	1.275406	4.237972	1.900306	3.362859	3.647421	...	0.973811
3	2.307073	1.658853	1.443538	2.208859	2.226486	0.780790	1.967043	0.924908	2.970700	2.551466	...	0.520354

d. 예측 사용자 - 아이템 평점 행렬 정보를 이용해 개인화 영화 추천

→ `get_unseen_movies()` 함수,
`recommen_movie_by_userid()`

→ 앞 절의 아이템 기반 협업 필터링 결과와 추천
된 영화가 많이 다름

```
# 사용자가 관람하지 않은 영화명 추출
unseen_list = get_unseen_movies(ratings_matrix, 9)

# 잠재 요인 협업 필터링으로 영화 추천
recomm_movies = recomm_movie_by_userid(ratings_pred_matrix, 9, unseen_list, top_n=10)

# 평점 데이터를 DataFrame으로 생성.
recomm_movies = pd.DataFrame(data=recomm_movies.values, index=recomm_movies.index,
                              columns=['pred_score'])
recomm_movies
```

	pred_score
title	
Rear Window (1954)	5.704912
South Park: Bigger, Longer and Uncut (1999)	5.451100
Rounders (1998)	5.298363
Blade Runner (1982)	5.244951
Roger & Me (1989)	5.191902
Gattaca (1997)	5.183179
Ben-Hur (1959)	5.130463
Rosencrantz and Guildenstern Are Dead (1990)	5.087375
Big Lebowski, The (1998)	5.038960
Star Wars: Episode V - The Empire Strikes Back (1980)	4.989501